

CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Disciplina: Algoritmos e Programação Orientada a Objetos II (APO2)

Sistema de Gestão de Biblioteca MARC21

Plataforma Jakarta EE — Apache Tomcat 9.0

Integrantes:

Hálan Sousa, Naiany Sobral, Maria Machado

Sumário

1	Objetivo do Sistema	3
2	Equipe e Funções dos Integrantes	4
3	Requisitos do Sistema	4
3.1	Requisitos Tecnológicos	4
3.2	Requisitos Funcionais	4
3.3	Requisitos Não-Funcionais	6
4	Modelagem do Sistema	6
4.1	Diagrama de Classes	6
4.2	Arquitetura de Fluxo de Dados	7
4.3	Diagrama Entidade-Relacionamento	7
4.4	Diagrama de Caso de Uso	8
4.5	Diagrama de Sequência: Solicitação e Aprovação de Empréstimo	8
5	Relatório de Testes de Endpoints	9
5.1	Endpoint: POST /api/login	9
5.2	Endpoint: POST /api/cadastro	10
5.3	Endpoint: GET /api/ativar	10
5.4	Endpoint: POST /api/recuperar-senha	11
5.5	Endpoint: POST /api/emprestimos	11
5.6	Endpoint: GET/POST /api/admin/usuarios	13
5.7	Endpoint: GET/POST /api/admin/configuracoes	13
6	Relatório de Testes de Funcionalidades	14
6.1	Funcionalidade: Cadastro e Ativação de Conta	14
6.2	Funcionalidade: Autenticação e Controle de Acesso	15
6.3	Funcionalidade: Catalogação e Filtros do Acervo (MARC21)	15
6.4	Funcionalidade: Controle de Empréstimos e Devoluções	16

6.5	Funcionalidade: Renovação Online e Regras de Negócio	16
6.6	Funcionalidade: Configurações do Sistema	16
6.7	Funcionalidade: Perfil e Controle de Usuários	16

1 Objetivo do Sistema

O presente trabalho tem como objetivo o desenvolvimento de um **Sistema de Gestão de Biblioteca** utilizando a plataforma **Jakarta EE (Java EE 8)**, com servidor de aplicação **Apache Tomcat 9.0**, banco de dados **MySQL 8.0** e interface web construída com **Bootstrap 5.0** e requisições assíncronas via **jQuery/AJAX**.

O sistema foi concebido para atender as necessidades operacionais de uma biblioteca de médio porte, contemplando as seguintes funções principais:

- **Catálogo do Acervo Bibliográfico:** Registro de obras físicas com suporte ao padrão internacional **MARC21** (*Machine-Readable Cataloging*), permitindo a indexação de metadados bibliográficos como título (Tag 245), autor (Tag 100), ISBN (Tag 020) e assuntos (Tag 650), entre outros.
- **Controle de Solicitações, Empréstimos e Devoluções:** Gerenciamento do ciclo de vida completo do empréstimo de livros físicos, incluindo solicitação pelo leitor, aprovação ou recusa pelo administrador, registro de retirada, controle de prazos (prazo padrão de 7 dias corridos), registro de devoluções e cálculo automático de multas por atraso.
- **Renovação Online:** Permite que leitores autenticados renovem o prazo de seus empréstimos de forma assíncrona, respeitando o limite máximo de renovações configurável pelo administrador.
- **Gestão de Usuários e Controle de Acesso (RBAC):** O sistema implementa o padrão de segurança por Controle de Acesso Baseado em Papéis (*Role-Based Access Control*), diferenciando os perfis de **Administrador** e **Cliente (Leitor)**.
- **Validação de Cadastro por E-mail:** Usuários recém-cadastrados precisam confirmar seu endereço de e-mail através de um link de ativação gerado com token único (UUID), enviado via protocolo **SMTP** com suporte ao serviço de testes **Mailtrap**.
- **Listas de Leitura Personalizadas:** Leitores podem criar coleções pessoais de livros de interesse, como pastas de favoritos.
- **Configurações Operacionais Dinâmicas:** Administradores podem ajustar parâmetros como o valor da multa diária e o limite de renovações diretamente pelo painel administrativo, sem necessidade de intervenção no código.

O projeto foi desenvolvido com base no **Documento de Diretrizes Arquiteturais** elaborado previamente, seguindo os princípios **SOLID** — em especial o Princípio

da Substituição de Liskov (LSP) na hierarquia de herança do acervo (`ItemAcervo` → `LivroFisico`, `EBook`) e o Princípio Aberto/Fechado (OCP) na extensibilidade das classes de persistência (`BaseDAO`).

A arquitetura segue o padrão **MVC (Model–View–Controller)**, separando claramente as responsabilidades entre as camadas de modelo (Entidades e DAOs), controle (Servlets HTTP) e visão (páginas JSP com Bootstrap).

2 Equipe e Funções dos Integrantes

A equipe responsável pelo desenvolvimento deste projeto é composta pelos seguintes integrantes:

Tabela 1: Integrantes e participação no projeto

Integrantes	Participação
Hálan Sousa, Naiany Sobral e Maria Machado	Todos os integrantes participaram de forma conjunta em todas as etapas do desenvolvimento do projeto, incluindo levantamento de requisitos, modelagem do banco de dados, implementação do back-end e do front-end, integração entre as camadas, testes, documentação e validação das funcionalidades, sem divisão fixa de responsabilidades.

3 Requisitos do Sistema

3.1 Requisitos Tecnológicos

O projeto foi desenvolvido atendendo integralmente os seguintes requisitos tecnológicos obrigatórios:

3.2 Requisitos Funcionais

- **RF01** — Cadastro de Usuário: Permite o auto-cadastro de leitores informando nome, e-mail e senha.
- **RF02** — Ativação de Conta: Envio de e-mail assíncrono com token exclusivo para validação do cadastro.

Tabela 2: Requisitos tecnológicos e status de atendimento

Tecnologia	Aplicação no Projeto	Status
Jakarta EE / Tomcat 9+	Servidor de aplicação principal; API de Servlets 4.0	✓
MySQL Server 8.0+	Banco de dados relacional; scripts em <code>ddl.sql</code> e <code>dml.sql</code>	✓
jQuery	Requisições AJAX, manipulação DOM, máscaras de entrada	✓
Bootstrap 5.0	Layout responsivo, componentes visuais, modais	✓
AJAX	Chamadas assíncronas para todos os endpoints	✓
Endpoints JSON	Retorno <code>application/json</code> em todos os Servlets	✓
Testes Postman	Relatório de testes na Seção 5	✓
Servlets como endpoints	Todos os endpoints são Servlets mapeados por anotações <code>@WebServlet</code> ; o <code>web.xml</code> define a página inicial	✓
JSP para views	Todas as páginas são construídas em JSP com JSTL	✓

- **RF03** — Autenticação: Sistema de login seguro diferenciando perfis por papéis através de filtros.
- **RF04** — Recuperação de Senha: Gera uma senha provisória aleatória e envia ao e-mail do usuário ativo.
- **RF05** — Consulta ao Acervo: Filtros e buscas de livros e e-books por título, autor ou assunto.
- **RF06** — Detalhes do Item: Visualização completa dos metadados bibliográficos estruturados no padrão MARC21.
- **RF07** — Solicitação de Empréstimo: Leitores com conta ativa e sem débitos podem solicitar a retirada de livro físico disponível, ficando a solicitação pendente de aprovação administrativa.
- **RF08** — Renovação Assíncrona: Extensão do prazo de entrega via painel do leitor, respeitando limites operacionais.
- **RF09** — Listas de Interesse: Criação e gerenciamento de coleções de favoritos de livros físicos pelo leitor.
- **RF10** — Painel de Controle (Dashboard): Indicadores gráficos e contadores quantitativos específicos para administradores e leitores.
- **RF11** — Gerenciamento de Acervo: Cadastro, edição e exclusão de livros físicos e e-books, incluindo campos MARC21.
- **RF12** — Gerenciamento de Usuários: Listagem de usuários, alteração de perfil (admin/leitor) e inativação.

- **RF13** — Controle de Empréstimos: Aprovação ou recusa de solicitações pelo administrador, registro de devolução e visualização de pendências.
- **RF14** — Relatório de Inadimplência: Listagem de empréstimos em atraso com cálculo de multa acumulada.
- **RF15** — Configurações do Sistema: Ajuste de parâmetros operacionais (prazo padrão, valor de multa, limite de renovações).

3.3 Requisitos Não-Funcionais

- **RNF01 — Segurança:** Senhas armazenadas com hash SHA-256. Sessões gerenciadas pelo Tomcat com filtro de autenticação em todas as rotas protegidas.
- **RNF02 — Usabilidade:** Interface responsiva (Bootstrap 5) com feedback imediato ao usuário via mensagens AJAX.
- **RNF03 — Manutenibilidade:** Arquitetura em camadas (MVC) com padrão DAO e herança (BaseDAO), seguindo os princípios SOLID.
- **RNF04 — Portabilidade:** Empacotado como `.war` compatível com qualquer container Servlet 4.0+ (Tomcat 9/10, WildFly, GlassFish).

4 Modelagem do Sistema

4.1 Diagrama de Classes

O sistema foi projetado seguindo uma hierarquia de herança clara para o acervo bibliográfico. A classe abstrata `ItemAcervo` define o contrato comum para todos os itens do acervo, sendo estendida pelas classes concretas `LivroFisico` e `EBook`.

```
ItemAcervo <|-- LivroFisico
ItemAcervo <|-- EBook
Usuario "1" ---- "N" Emprestimo
LivroFisico "1" ---- "N" Emprestimo
LivroFisico "1" ---- "N" CampoMARC
Usuario "1" ---- "N" ListaLeitura
ListaLeitura "N" ---- "N" LivroFisico
BaseDAO <|-- UsuarioDAO / LivroFisicoDAO / EmprestimoDAO / ConfiguracaoDAO
```

As principais relações de herança e associação são:

- **ItemAcervo (Abstrata)** $\leftarrow$ LivroFisico: Representa obras físicas do acervo com controle de quantidade disponível em estoque.
- **ItemAcervo (Abstrata)** $\leftarrow$ EBook: Representa livros digitais com URL de acesso e sem restrição de quantidade.
- **BaseDAO (Abstrata)** $\leftarrow$ Todos os DAOs: Centraliza a lógica de obtenção de conexão JDBC.
- **Emprestimo** associa **Usuario** e **LivroFisico**: Representa a solicitação e, após aprovação administrativa, o contrato de retirada física.

4.2 Arquitetura de Fluxo de Dados

O fluxo de execução das requisições assíncronas (AJAX) segue de forma padronizada o seguinte caminho:

```
Navegador  $\xrightarrow{\text{HTTP Request}}$  AutenticacaoFilter  $\rightarrow$  XServlet  $\rightarrow$  XServletDAO  $\rightarrow$  MySQL
MySQL  $\xrightarrow{\text{ResultSet}}$  XServletDAO  $\rightarrow$  Gson.toJson()  $\xrightarrow{\text{JSON}}$  jQuery/AJAX  $\rightarrow$  DOM
```

4.3 Diagrama Entidade-Relacionamento

O modelo relacional do banco de dados organiza os cadastros principais e as relações operacionais de empréstimo, metadados MARC21 e listas de leitura.

usuarios (id PK)

1 ---- N empréstimos (usuario_id FK)
 1 ---- N listas_leitura (usuario_id FK)

livros (id PK)

1 ---- N empréstimos (livro_id FK)
 1 ---- N campos_marc (livro_id FK)
 N ---- N listas_leitura via listas_leitura_livros

empréstimos

id PK

usuario_id FK -> usuarios.id

livro_id FK -> livros.id

status: SOLICITADO, ATIVO, DEVOLVIDO ou RECUSADO

campos_marc

id PK

livro_id FK -> livros.id

listas_leitura_livros

lista_leitura_id FK -> listas_leitura.id

livro_id FK -> livros.id

PK composta (lista_leitura_id, livro_id)

configuracoes_globais

id PK

4.4 Diagrama de Caso de Uso

Visitante -> Consultar acervo publico

Visitante -> Cadastrar usuario

Visitante -> Ativar conta por e-mail

Visitante -> Recuperar senha

Usuario autenticado -> Atualizar perfil

Usuario autenticado -> Solicitar emprestimo

Usuario autenticado -> Renovar emprestimo

Usuario autenticado -> Criar lista ou listas de livros

Administrador -> Gerenciar acervo

Administrador -> Gerenciar usuarios e niveis

Administrador -> Aprovar ou recusar solicitacao de emprestimo

Administrador -> Registrar devolucao

Administrador -> Atualizar configuracoes

4.5 Diagrama de Sequência: Solicitação e Aprovação de Empréstimo

Leitor -> dashboard.jsp: clica em Solicitar Empréstimo

dashboard.jsp -> EmpréstimoServlet: POST /api/emprestimos

EmpréstimoServlet -> AutenticacaoFilter: valida sessão

EmpréstimoServlet -> LivroFisicoDAO: consulta disponibilidade

EmpréstimoServlet -> EmpréstimoDAO: registra status SOLICITADO

EmpréstimoServlet -> dashboard.jsp: retorna JSON aguardando aprovação

Administrador -> admin.jsp: clica em Aprovar

```
admin.jsp -> EmprestimoServlet: POST /api/emprestimos acao=aprovar
EmprestimoServlet -> LivroFisicoDAO: decrementa estoque
EmprestimoServlet -> EmprestimoDAO: altera status para ATIVO
EmprestimoServlet -> admin.jsp: retorna JSON sucesso
```

5 Relatório de Testes de Endpoints

Os endpoints da aplicação foram testados utilizando a ferramenta **Postman** (versão 10+). Para cada endpoint, foram realizados testes com dados válidos e dados inválidos para verificar o comportamento esperado da API.

Configuração do ambiente de teste:

- URL Base: `http://localhost:8080/biblioteca`
- Servidor: Apache Tomcat 9.0 (local)
- Banco de Dados: MySQL 8.0 (local)

5.1 Endpoint: POST /api/login

Descrição: Autentica o usuário e retorna os dados da sessão em JSON.

Teste 1.1 — Login de Administrador com sucesso:

```
1 POST http://localhost:8080/biblioteca/api/login
2 Content-Type: application/x-www-form-urlencoded
3
4 email=admin@biblioteca.com&senha=admin123
```

Listing 1: Requisição: POST /api/login (Administrador)

```
1 {
2     "sucesso": true,
3     "mensagem": "Login realizado com sucesso.",
4     "usuario": {
5         "id": 1,
6         "nome": "Administrador",
7         "perfil": "ADMIN",
8         "redirect": "/biblioteca/admin.jsp"
9     }
10 }
```

Listing 2: Resposta esperada (HTTP 200 OK)

Teste 1.2 — Login com credenciais inválidas:

```
1 {  
2     "sucesso": false,  
3     "mensagem": "E-mail ou senha invalidos."  
4 }
```

Listing 3: Resposta esperada (HTTP 401 Unauthorized)

5.2 Endpoint: POST /api/cadastro

Descrição: Realiza o cadastro de um novo leitor no sistema.

```
1 POST http://localhost:8080/biblioteca/api/cadastro  
2 Content-Type: application/x-www-form-urlencoded  
3  
4 nome=Fulano+da+Silva&email=fulano@gmail.com&senha=senha123
```

Listing 4: Requisição: POST /api/cadastro

```
1 {  
2     "sucesso": true,  
3     "mensagem": "Cadastro realizado com sucesso! Verifique seu e-  
4 mail para ativar a conta."
```

Listing 5: Resposta esperada (HTTP 201 Created)

Teste 2.2 — Tentativa de cadastro com e-mail duplicado:

```
1 {  
2     "sucesso": false,  
3     "mensagem": "Este e-mail ja esta cadastrado."  
4 }
```

Listing 6: Resposta esperada (HTTP 409 Conflict)

5.3 Endpoint: GET /api/ativar

Descrição: Ativa a conta do usuário através do token UUID enviado por e-mail.

```
1 GET http://localhost:8080/biblioteca/api/ativar?email=fulano@gmail.  
com&token=9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d
```

Listing 7: Requisição: GET /api/ativar

```
1 {
2     "sucesso": true,
3     "mensagem": "Conta ativada com sucesso! Voce ja pode fazer login"
4 }
```

Listing 8: Resposta esperada (HTTP 200 OK)

5.4 Endpoint: POST /api/recuperar-senha

Descrição: Envia uma senha temporária por e-mail se a conta estiver ativa.

```
1 POST http://localhost:8080/biblioteca/api/recuperar-senha
2 Content-Type: application/x-www-form-urlencoded
3
4 email=fulano@gmail.com
```

Listing 9: Requisição: POST /api/recuperar-senha

```
1 {
2     "sucesso": true,
3     "mensagem": "Se o e-mail existir e a conta estiver ativa,
4     enviaremos uma senha temporaria."
```

Listing 10: Resposta esperada (HTTP 200 OK)

5.5 Endpoint: POST /api/emprestimos

Descrição: Registra uma solicitação de empréstimo para um livro físico disponível. O estoque somente é alterado depois da aprovação administrativa.

```
1 POST http://localhost:8080/biblioteca/api/emprestimos
2 Content-Type: application/x-www-form-urlencoded
3 Cookie: JSESSIONID={token_da_sessao_leitor}
4
5 acao=cadastrar&livroId=5
6
```

Listing 11: Requisição: POST /api/emprestimos

```
1 {
2     "sucesso": true,
3     "mensagem": "Solicitacao enviada com sucesso! Aguarde a
4     aprovacao do administrador para retirar o livro."
```

```
4      }  
5
```

Listing 12: Resposta esperada (HTTP 200 OK)

Teste 5.1 — Admin aprova solicitação de empréstimo:

```
1      POST http://localhost:8080/biblioteca/api/emprestimos  
2      Content-Type: application/x-www-form-urlencoded  
3      Cookie: JSESSIONID={token_da_sessao_admin}  
4  
5      acao=aprovar\&emprestimoId=10  
6
```

Listing 13: Requisição: POST /api/emprestimos acao=aprovar

```
1      {  
2          "sucesso": true,  
3          "mensagem": "Solicitacao aprovada. O emprestimo foi iniciado  
4      ..."  
5      }
```

Listing 14: Resposta esperada (HTTP 200 OK)

Teste 5.2 — Admin recusa solicitação de empréstimo:

```
1      POST http://localhost:8080/biblioteca/api/emprestimos  
2      Content-Type: application/x-www-form-urlencoded  
3      Cookie: JSESSIONID={token_da_sessao_admin}  
4  
5      acao=recusar\&emprestimoId=10  
6
```

Listing 15: Requisição: POST /api/emprestimos acao=recusar

```
1      {  
2          "sucesso": true,  
3          "mensagem": "Solicitacao recusada sem alterar o estoque."  
4      }  
5
```

Listing 16: Resposta esperada (HTTP 200 OK)

Teste 5.3 — Usuário não autenticado tentando acessar:

```
1      {  
2          "sucesso": false,  
3          "mensagem": "Acesso negado. Faca login primeiro."  
4      }
```

5

Listing 17: Resposta esperada (HTTP 401 Unauthorized)

Teste 5.4 — Leitor tentando acessar endpoint de Admin:

```
1      {
2          "sucesso": false,
3          "mensagem": "Acesso negado. Permissao insuficiente."
4      }
5
```

Listing 18: Resposta esperada (HTTP 403 Forbidden)

5.6 Endpoint: GET/POST /api/admin/usuarios

Descrição: Lista usuários cadastrados e permite ao administrador alterar perfil e status.

```
1      POST http://localhost:8080/biblioteca/api/admin/usuarios
2      Content-Type: application/x-www-form-urlencoded
3      Cookie: JSESSIONID={token_da_sessao_admin}
4
5      id=2\&tipoUsuario=CLIENTE\&ativo=true
6
```

Listing 19: Requisição: POST /api/admin/usuarios

```
1      {
2          "sucesso": true,
3          "mensagem": "Usuario atualizado com sucesso."
4      }
5
```

Listing 20: Resposta esperada (HTTP 200 OK)

5.7 Endpoint: GET/POST /api/admin/configuracoes

Descrição: Consulta e atualiza parâmetros operacionais do sistema.

```
1      POST http://localhost:8080/biblioteca/api/admin/configuracoes
2      Content-Type: application/x-www-form-urlencoded
3      Cookie: JSESSIONID={token_da_sessao_admin}
4
5      multaDiaria=2.50\&limiteRenovacoes=3\&prazoEmprestimo=7
6
```

Listing 21: Requisição: POST /api/admin/configuracoes

```

1      {
2          "sucesso": true,
3          "mensagem": "Parametros operacionais atualizados com sucesso"
4      }
5  
```

Listing 22: Resposta esperada (HTTP 200 OK)

6 Relatório de Testes de Funcionalidades

Esta seção documenta os testes realizados nas funcionalidades completas do sistema, verificando o fluxo de ponta a ponta (interface

→ AJAX

→ Servlet

→ DAO

→ MySQL).

6.1 Funcionalidade: Cadastro e Ativação de Conta

Objetivo: Verificar o fluxo completo de registro de um novo usuário, incluindo o recebimento e validação do e-mail de ativação via Mailtrap.

Tabela 3: Casos de teste — Cadastro e Ativação

CT#	Ação	Resultado Esperado	Status
CT-01	Preencher todos os campos do formulário de cadastro com dados válidos e submeter.	Mensagem de sucesso exibida via AJAX; e-mail de ativação enviado ao Mailtrap.	✓
CT-02	Submeter o formulário com o campo e-mail em branco.	Validação HTML5/jQuery impede envio; mensagem de erro “Campo obrigatório”.	✓
CT-03	Tentar cadastrar com um e-mail já existente no banco.	Retorno JSON com erro 409; mensagem “E-mail já cadastrado” exibida na tela.	✓
CT-04	Acessar o link de ativação recebido no Mailtrap.	Conta ativada; redirecionamento para login com mensagem de confirmação.	✓

6.2 Funcionalidade: Autenticação e Controle de Acesso

Tabela 4: Casos de teste — Login e RBAC

CT#	Ação	Resultado Esperado	Status
CT-05	Efetuar login com credenciais de Administrador.	Usuário autenticado com sucesso e redirecionado para <code>admin.jsp</code> .	✓
CT-06	Efetuar login com credenciais de Leitor (ativo).	Usuário autenticado com sucesso e redirecionado para <code>dashboard.jsp</code> .	✓
CT-07	Tentar logar com conta existente mas não ativada por e-mail.	Mensagem informando a necessidade de validação de conta antes do primeiro acesso.	✓
CT-08	Inserir senha incorreta para e-mail cadastrado.	Erro capturado; mensagem “E-mail ou senha inválidos” sem expor detalhes estruturais.	✓
CT-09	Leitor tenta digitar diretamente a URL <code>/admin.jsp</code> .	Bloqueado por <code>AutenticacaoFilter</code> e redirecionado ao <code>index</code> com aviso de restrição.	✓

6.3 Funcionalidade: Catalogação e Filtros do Acervo (MARC21)

Tabela 5: Casos de teste — Acervo e MARC21

CT#	Ação	Resultado Esperado	Status
CT-10	Admin insere um livro físico mapeando as tags MARC21 (020, 100, 245).	Dados distribuídos corretamente entre as tabelas correspondentes no MySQL.	✓
CT-11	Admin tenta cadastrar item sem preencher a Tag 245 (Título obrigatório).	Alerta de validação impede o envio; transação não enviada ao banco de dados.	✓
CT-12	Usuário realiza busca rápida por termo presente na tag de assunto (Tag 650).	Listagem atualizada dinamicamente via AJAX exibindo apenas os registros correspondentes.	✓
CT-13	Leitor clica em visualizar detalhes de uma obra física do acervo.	Modal abre exibindo a ficha catalográfica formatada com os rótulos legíveis das Tags MARC21.	✓

Tabela 6: Casos de teste — Empréstimos

CT#	Ação	Resultado Esperado	Status
CT-14	Leitor solicita empréstimo de livro com estoque disponível.	Registro criado com status SOLICITADO ; estoque permanece inalterado até aprovação administrativa.	✓
CT-15	Leitor tenta solicitar livro que está com estoque zerado (indisponível).	Botão desabilitado na interface; requisição manual direta via endpoint devolve erro.	✓
CT-16	Admin aprova uma solicitação pendente.	Status alterado para ATIVO ; data de retirada atualizada; estoque decrementado em 1 unidade.	✓
CT-17	Admin recusa uma solicitação pendente.	Status alterado para RECUSADO ; estoque permanece inalterado.	✓
CT-18	Admin registra devolução dentro do prazo regulamentar.	Status do empréstimo alterado para DEVOLVIDO ; estoque incrementado em +1 unidade.	✓
CT-19	Admin registra devolução de empréstimo com 3 dias de atraso.	Sistema calcula automaticamente o valor da multa com base na taxa diária operacional vigente.	✓

Tabela 7: Casos de teste — Renovações

CT#	Ação	Resultado Esperado	Status
CT-20	Leitor solicita renovação de empréstimo ativo dentro do prazo regular.	Data de devolução estendida por mais 7 dias; contador incrementado.	✓
CT-21	Leitor tenta renovar uma solicitação ainda não aprovada.	Renovação recusada; apenas empréstimos com status ATIVO podem ser renovados.	✓
CT-22	Leitor tenta renovar um empréstimo já em atraso (vencido).	Renovação recusada; sistema exige a devolução imediata do item físico e quitação do débito.	✓
CT-23	Leitor tenta realizar a 4ª renovação consecutiva (limite configurado = 3).	Ação bloqueada pelo servidor; mensagem de erro indica limite de renovações atingido.	✓

6.4 Funcionalidade: Controle de Empréstimos e Devoluções

6.5 Funcionalidade: Renovação Online e Regras de Negócio

6.6 Funcionalidade: Configurações do Sistema

6.7 Funcionalidade: Perfil e Controle de Usuários

Tabela 8: Casos de teste — Configurações

CT#	Ação	Resultado Esperado	Status
CT-24	Admin altera o valor da multa diária para R\$ 2,50.	Novo valor persistido no banco; próximo cálculo de multa usa o valor atualizado.	✓
CT-25	Admin altera o limite de renovações para 1 (mínimo).	Leitores que já renovaram uma vez não podem renovar novamente.	✓
CT-26	Leitor tenta acessar GET /api/admin/configuracoes.	HTTP 403 retornado pelo AutenticacaoFilter.	✓

Tabela 9: Casos de teste — Perfil e Usuários

CT#	Ação	Resultado Esperado	Status
CT-27	Leitor atualiza nome e senha no painel.	Dados persistidos; sessão atualizada; senha gravada com hash SHA-256.	✓
CT-28	Admin altera papel de usuário de 'Leitor' para 'Admin'.	Permissões atualizadas instantaneamente no banco de dados.	✓
CT-29	Admin inativa um usuário inatimplente crônico.	Usuário não consegue realizar login e tem acessos cortados.	✓

Referências

- [1] Bootstrap Core Team. Bootstrap 5 — the world’s most popular front-end open source toolkit, 2024.
- [2] P. J. Deitel and H. M. Deitel. *Java: Como Programar*. Pearson, São Paulo, 10 edition, 2017.
- [3] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Reading, MA, 1994.
- [4] D. Geary and C. S. Horstmann. *Core JavaServer Faces*. Prentice Hall, 3 edition, 2010.
- [5] Google. Gson user guide, 2024.
- [6] C. S. Horstmann. *Core Java, Volume II – Advanced Features*. Prentice Hall, 11 edition, 2019.
- [7] jQuery Foundation. jquery api documentation, 2024.
- [8] Library of Congress. Marc 21 format for bibliographic data, 2024.
- [9] R. C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, Upper Saddle River, NJ, 2008.

- [10] Mozilla Foundation. Mdn web docs – fetch api, 2024.
- [11] Oracle Corporation. Mysql 8.0 reference manual, 2024.
- [12] The Apache Software Foundation. Apache tomcat 9.0 documentation, 2024.